

In Defense of Euler Angles in Game Programming

Matthew Whiting
University of Southern California
Los Angeles, California, USA
whitingm@usc.edu

Abstract

When teaching three-dimensional rotations, instruction often moves quickly from Euler angles to quaternions. This is even the approach in our Video Game Programming course at the University of Southern California. The prevailing rationale repeated by both educators and those in industry is that we avoid Euler angles due to gimbal lock and the associated loss of a degree of freedom. However, an analysis of the typical problems in game development shows that in most cases, we do not actually need three degrees of freedom. Thus, in many cases Euler angles can be a simpler, more convenient approach than quaternions. In fact, many typical rotational problems in games are inherently constrained to two degrees of freedom. In these cases, quaternion interpolation (spherical linear interpolation or *slerp*) can produce undesirable effects, whereas Euler angle interpolation produces the desired result. Common examples include target tracking scenarios defined by azimuth and elevation (yaw and pitch), as well as camera systems in which player input is explicitly divided along these axes.

This paper argues that Euler angles should be presented as a first-class representation for constrained rotational problems. By teaching both Euler angles and quaternions as complementary tools, and by emphasizing practical techniques such as angle extraction using *atan2*, educators can better equip students to select rotation representations that minimize complexity and produce predictable, application-appropriate motion.

CCS Concepts

• **Applied computing** → **Education**; **Computer games**; • **Social and professional topics** → *Computer science education*; • **Computing methodologies** → *Procedural animation*.

Keywords

Euler Angles, Quaternions, Gimbal Lock, Undergraduate Education, Game Programming, Altazimuth

ACM Reference Format:

Matthew Whiting. 2026. In Defense of Euler Angles in Game Programming. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3799818.3812087>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

SIGGRAPH Talks '26, Los Angeles, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-XXXX-X/2026/07

<https://doi.org/10.1145/3799818.3812087>

1 Introduction

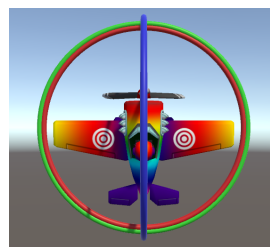
When three-dimensional coordinate systems are first introduced, Euler angles are often presented as a natural and intuitive representation of orientation. As human beings, this is how we naturally navigate the world. For all practical purposes, we move around in the horizontal plane, turning left or right (yaw). When the need arises, we can look up or down (pitch), and although we are certainly capable of it, tilting sideways (roll) is rare in daily life.

In the context of three-dimensional graphics, however, instruction often shifts quickly toward quaternions. Euler angles are commonly characterized as problematic due to issues such as gimbal lock and poor interpolation behavior. As a result, quaternions are widely regarded as the superior and safer representation. This prevailing perspective has led many graphics and gameplay engineers to overlook scenarios in which Euler angles, by virtue of how they interpolate, may in fact be the more appropriate choice.

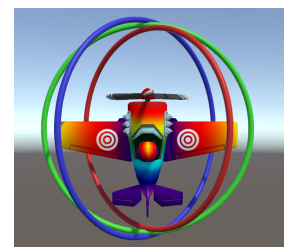
2 Gimbal Lock

The term *gimbal lock* originates from physical mechanisms composed of concentric rotational frames, or gimbals. With three such rotational axes, a system can represent arbitrary orientation in three-dimensional space. However, at certain configurations, two of these axes may become aligned, reducing the system's ability to distinguish rotations about those axes.

In practice, this alignment does not imply a true loss of rotational capability. For example, an aircraft oriented vertically still retains full control authority through its control surfaces and can execute pitch, roll, and yaw maneuvers in body space. The limitation instead arises in how those rotations are expressed relative to a fixed reference frame. At a pitch of 90°, this alignment introduces a discontinuity in the Euler angle parameterization. This discontinuity is the source of the phenomenon commonly referred to as gimbal lock. (Figure 1) Importantly, in a computational context, Euler angles are a mathematical representation rather than a physical mechanism. There is no inherent constraint preventing the representation of arbitrary orientations, nor is any degree of rotational freedom



(a) Yaw And Roll Coincide



(b) Equivalent Orientation

Figure 1: Gimbal Lock

lost. What is lost is the convenience of a smooth and continuous mapping between numeric parameters and spatial orientation.

While these discontinuities at the poles of the parameter space can be inconvenient, these singularities are an inherent characteristic in certain physical mechanical systems being modeled. In these cases, the singularity may not be resolved by switching to quaternions. Alternatively, there are practical approaches to resolve the singularity while relying primarily on Euler angle calculations. It is therefore necessary to consider the underlying rotational behavior being modeled and the constraints imposed by the chosen parameterization.

3 Two Degrees of Freedom

In many applications, the rotational behavior being modeled is inherently constrained to two degrees of freedom. A common example is a seated observer tracking a moving object within their field of view. While the object may move freely in three-dimensional space, the observer's head motion can be described entirely by azimuth (rotation about the vertical axis or yaw) and elevation (rotation about the horizontal axis or pitch). This azimuth–elevation representation is widely used in physical hardware, particularly for tasks involving target tracking or directional alignment. Camera systems, antenna arrays, and telescope mounts frequently employ so-called *Az/El* or *altazimuth* configurations (Figure 2a).

When such constrained motion is instead modeled using quaternions, unintended behavior can arise. For example, Figure 2b illustrates a laser turret in a video game executing a spherical linear interpolation (slerp) between orientations in order to engage a



(a) Camera with Altazimuth Mount



(b) Turret Bending Over Backwards

Figure 2: Two Degrees of Freedom

target located behind it. While mathematically valid, this interpolation introduces rotational components that fall outside the intended azimuth–elevation constraint, producing motion that may be undesirable for the application.

4 Video Game Camera

Quaternions are frequently used by video game programmers to control virtual camera orientation [Bobic 1998]. While video games encompass a wide range of genres and camera systems, it is highly unusual for an in-game camera to intentionally rotate about the forward axis, commonly referred to as roll or *Dutch tilt*.

Quaternion-based spherical linear interpolation (slerp) does not inherently respect this constraint. When orientations are interpolated freely in quaternion space, roll may be introduced even when it is not desired, requiring additional logic to suppress it. In contrast, Euler angle representations allow pitch, yaw, and roll to be interpolated independently, ensuring that interpolation between orientations does not introduce unintended roll.

In a typical third-person camera system, the camera orbits around a player-controlled avatar, and the player has two axes of input to guide the camera. Horizontal input commands the camera to orbit the avatar in the horizontal plane (yaw), while input on the forward axis produces pitch. These two inputs are intentionally decoupled. When a yaw rotation of 180° is commanded, the expected behavior is a pure horizontal orbit with the pitch angle remaining unchanged. Quaternions are inherently more complex, and the resulting motion depends on what was expressed to create the quaternions. If the quaternion is generated as a shortest-path arc, slerp will traverse a great-circle arc over the poles while introducing unintended twist. Alternatively, a slerp between quaternions matching the equivalent Euler angles will be essentially indistinguishable from the Euler angle interpolation rendering the additional computational complexity completely unnecessary. A common concern arises when an orbiting video game camera approaches a pitch of 90° : whether the camera will become “locked” due to a perceived loss of a degree of freedom (gimbal lock). Rather than lost freedom, in practice, the underlying issue is understanding the intention behind the user's control input. Once the desired mapping from two-dimensional input to camera motion is defined, it can be implemented directly - often in terms of Euler angles. Figure 3 shows a third-person camera



Figure 3: Euler Angle Camera Crossing Poles Without Issue

allowed to rotate past 90° elevation using Euler angle calculations. This is not commonly allowed in video games, but it does not cause visual discontinuities or control issues.

5 Animation

While Euler angles can be effective in certain contexts, quaternions are generally preferred for jointed skeletal animation. Nevertheless, it is notable that Autodesk Maya defaults to Euler angles within its animation system [Autodesk 2015].

Euler angle interpolation operates in a decoupled manner, which can occasionally result in motion that appears unnatural. Additionally, interpolation near the poles of the parameter space can introduce discontinuities commonly associated with gimbal lock. In practice, however, skeletal joints are typically constrained to limited ranges of motion and seldom reach configurations in which these singularities occur. Consequently, artists are often able to work effectively using Euler angles without encountering significant issues.

Autodesk [Autodesk 2015] acknowledges the limitations of Euler angle interpolation including gimbal lock and flipping (where interpolation from -180° to 180° manifests as a 360° rotation). However, they also point out the advantage that Euler angles can be made to rotate from 0° to 720° in a single pair of keyframes while there is no way to represent that type of motion using quaternions.

6 atan2()

It is also worth highlighting the role of the *atan2* function in practical applications of Euler angles. Despite its convenience, this function is sometimes underutilized in graphics and game development workflows.

When converting Euler angles to position or direction vectors, the transformation is typically expressed using matrices composed of sine and cosine terms. Reversing this process might suggest the use of inverse sine and cosine functions; however, the inverse tangent function is often more practical. Most standard mathematics libraries provide a two-argument variant, *atan2*, which resolves angular values correctly across all four quadrants.

Extracting a rotation angle from a two-dimensional vector is a common operation in video game programming as with the azimuth-tracking problems outlined previously. Traditional approaches may rely on the arc cosine of a normalized vector component combined with additional logic to determine the correct sign. The *atan2* function provides a more direct and numerically robust solution:

$$\theta = \text{atan2}(v_y, v_x) \quad (1)$$

Likewise, the signed angular difference between vectors is commonly calculated by normalizing the vectors, performing a dot product, taking the arc cosine, and then a sign check on the z coordinate from a cross product. Leveraging *atan2*(), we have:

$$\Delta\theta = \text{atan2}(a_y, a_x) - \text{atan2}(b_y, b_x) \quad (2)$$

With separate inputs for the opposite and adjacent, *atan2*() solves the quadrant issue, but that can give the impression that *atan2*() is only suited to two-dimensional problems. Figure 4 illustrates how *atan2*() can be used to break a three-dimensional vector into

azimuth and elevation angles: the Euler angles suitable for use in an altazimuth tracking laser turret or a spherical orbiting camera.

$$d = \sqrt{P_x^2 + P_y^2 + P_z^2} \quad (3)$$

$$d_{xz} = \sqrt{P_x^2 + P_z^2} \quad (4)$$

$$\psi = \text{atan2}(P_x, P_z) \quad (5)$$

$$\theta = \text{atan2}(P_y, d_{xz}) \quad (6)$$

7 Conclusion

Quaternions are an effective and robust solution for representing orientations with three fully decoupled degrees of freedom, and they provide a convenient way to avoid the singularities associated with Euler angles. At the same time, it is important to recognize situations in which rotational motion is intentionally constrained. In these contexts, Euler angles may offer clearer semantics and more desirable behavior. Accordingly, engineers should be introduced to both representations as complementary tools, selecting the one most appropriate to the constraints and goals of the system being designed.

Acknowledgments

I would like to thank my colleagues at the University of Southern California, my peers in video game development, as well as all the students who have taken classes with us over the years.

References

- Autodesk. 2015. *Animated rotation in Maya*. Retrieved February 8, 2026 from https://help.autodesk.com/view/MAYALT/2015/ENU/?guid=Animation_Basics_Animated_rotation_in_Maya
- Nick Bobic. 1998. *Rotating Objects Using Quaternions*. Retrieved February 8, 2026 from <https://www.gamedeveloper.com/programming/rotating-objects-using-quaternions>

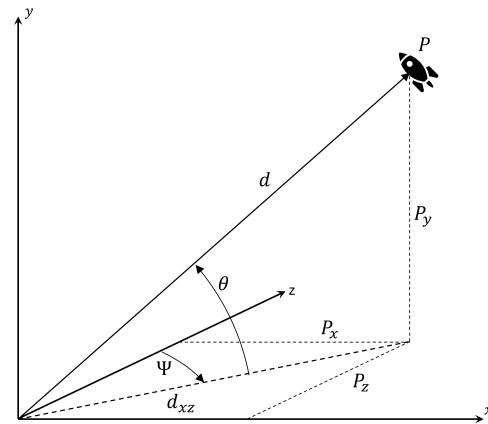


Figure 4: Evaluation of Euler Angles using *atan2*